

we aim to driver the following outcomes for our customers:

- By supporting Cells work, customers should experience improved reliability and compartmentalization of disruptions on GitLab.com.
- Expanding Service accounts will reduce the human touch points around credentials setup for automation use cases, bolstering customer's security posture and efficiency when using GitLab. At the same time, new Service account UI will allow them to easily setup, manage and revoke these higher privilege accounts providing better transparency and audit-ability. Combined with token management enhancements, customers will be able to confidently manage, enforce and mitigate access token related risks.
- Additional Enterprise user admin controls will result in reduced workload and improved security policy management for organizations while migration of Credential Inventory to SaaS will provide all administrators better visibility and control around credentials in use.
- We aim to improve and unify user provisioning setup for customers by expanding SCIM group sync support such that they don't need to rely on both SAML and SCIM for user provisioning and access management.

List of OKRs

Our OKRs are focused on:

- Improving support and help request experience
- Reducing the number of flaky or slow tests
- Career growth of Authentication team members
- Adopting Cells and sharing lessons from investigation spikes
- Roadmap completion around GCP Integration

Metrics the team tracks

In order to ensure we manage our technical debt while making progress on our roadmap, and addressing security vulnerabilities or bugs in time, the team tracks the following metrics:

- Engineering productivity metrics
 - MR rates across team to ensure changes are being delivered iteratively.
 - S1, S2 and S3 bugs and burndown rates to ensure that we are moving towards a downward trend on all 3. Currently no S1 are open, with a very small number of S2s.
 - Vulnerability due dates, in particular any that run the risk of hitting SLOs.
 - Error rates for Authentication API, workers and web workflows.
 - Infradev issues SLOs.
 - Maintainer-ship status for the group.
 - Domain knowledge areas and coverage across each one for bus factor tracking.
 - Feature flags associated with the group.
 - Flaky test attributed to the group.
- Product metrics
 - Adoption of auth integrations such as SAML, Identity providers, Group sync
 - Enterprise users claimed

What makes us different?

The Authentication group is a central piece to the GitLab product! While many groups focus on single area - like the repository view, or the merge request view, this group has a much broader impact on many areas. Because of this, there are some key topics that we keep on our mind more than other groups might:

Security vulnerability issues

Bypassing permissions and authentication mechanisms are, by nature, common security targets. We closely watch new security vulnerability issues and schedule them as quickly as possible based on their due date. For planning security issues we use the (filtered) milestone planning issue board. We expect all team members to be able to resolve security issues following the security developer workflow.

Code Review

Because this group works on components of the application that have a far-reaching impact, we take these extra steps in order to reduce our risk of a production incident:

1. Our team's merge requests should be assigned to another Auth team member for first review in order to build more institutional knowledge across the team. This review should be done as a reviewer. The Auth approval counts as the approval matching the role of the Auth Reviewer, e.g. having a Backend Review from Auth counts as a Backend Review. Once approved, the Auth Reviewer should request a review from a Maintainer from the appropriate maintainer category.
1. Auth merge requests will include a comment that needs answered before merging, "Should this be behind a feature flag?" This is an effort to remind engineers about feature flag usage, but also to challenge reasoning as to why changes do not need to be behind a feature flag.
1. Auth related merge requests require a review by an Auth Engineer. This is guarded by using the CODEOWNERS feature of GitLab.

Rollout and validation for token related features or destructive data updates

Due to the critical nature of tokens (PAT, PrAT and GrAT) and their associated bot accounts or memberships, a careful rollout process must be followed. This also extends to feature changes that update large datasets in the database. As part of feature completion, the DRI engineer must create a rollout plan and have it peer reviewed by at least 2 other engineers familiar with the domain.

The rollout plan can use any number of the following mechanisms to de-risk the changes:

1. Use of feature flags that can be set to test the feature out with a group or set of actors. If that is not possible, consider running the worker for a narrow batch of users until it has been validated and can be run on larger sets.
1. For database changes, evaluate the count of expected record changes before and after, the last update dates and the associated records to see whether a certain relationship may have been missed from implementation design.
1. Changes should be enabled on lower environments such as staging or staging-ref to verify expected functionality. As much as possible, attempt to replicate the data structures from GitLab.com on these lower environments.
1. Use of internal groups such as quality team or delivery teams which are production systems, but still internal. This would test the feature at a production level, ideally with no negative

impact, but still offer a layer of user testing prior to users.

1. Addition of audit events, and log data to leave a trail of operations that can offer traceability in case of unexpected changes.

It's important to consider that while maintainer reviews and test coverage are foundational requirements for feature completion, the rollout plan must be focused on being ready for unknown-unknowns i.e. Mechanisms to identify, prevent and mitigate if the feature doesn't perform as believed at the time of implementation.

Lastly, due to the direct impact of authentication change on the ability to access and use GitLab, it is better to be slower in the rollout than rush to meet milestone dates in almost all situations. Based on our value to place customers first, utilize all available tools to test and rollout features in a graduated manner. If in doubt refer to our prioritization framework and apply this here - with security and preventing data loss our main priorities.

How we work

- In accordance with our GitLab values.
- Transparently: nearly everything is public, we record/livestream meetings whenever possible.
- We get a chance to work on the things we want to work on.
- Everyone can contribute; no silos.
 - The goal is to have product give engineering and design the opportunity to be involved with direction and issue definition from the very beginning.
- We do an optional, asynchronous daily stand-up in our group stand-up channel:
 - SSCS:Authentication gsscsauthenticationdaily

With our counterparts

You are encouraged to work as closely as needed with stable counterparts. We include quality engineering and application security counterparts prior to a release kickoff and as-needed during code reviews or issue concerns.

Quality engineering is included in our workflow via the Quad Planning Process and is responsible for bug prioritization during release planning. They are also the DRI when it comes to adding new end-to-end tests, though anyone can contribute. Here are some examples of when to engage with your counterpart:

- Seeking Quality's opinion if an MR would introduce flakiness in existing end-to-end tests
- You need help deciding the best way to fix a test
- Guidance on setting up your local environment to properly test

Application Security will be involved in our security issue workflow and should participate in other feature, bug, or maintenance issues before they are scheduled if we need to notate any concerns or potential risks that we should be aware of. Here are some examples of engaging with your counterpart:

- If you want to confirm whether something should be kept confidential
- Confirming whether or not something is considered a vulnerability
- Collaborating on feature proposals to see if they have any security implications

Support priority requests

As the primary interface between customers and engineering team, support team has insights into product and process improvements that will help reduce friction for our customers along with the need for manual intervention by the support team. Feature or enhancement requests such as these are labelled with Support Priority and surfaced by support team counterparts during milestone planning, in the group's planning issue for the respective milestone. At least once a year, when planning for yearly roadmaps, these are also matched against the impact on customers and support. These are then prioritized by the group based on available capacity for each milestone.

Keeping yourself informed

- Key channels of interest are engineering-fyi, team-member-updates, gsscsauthentication, secsection, ceo, cto
- We create a weekly issue to inform the team members about the company or team updates, to share important links or to be informed about the team availability. Creation of the issue is the responsibility of an Engineering Manager, who can use an issue template located in the SSCS/Auth repo. All weekly updates can be found in the project issue list filtered by weekly update label.

MR review requests

CODEOWNER file contains commons code areas that can potentially affect authentication related behaviours and is used to request group::authentication reviews during development. The request source can be directly related to an authentication feature either coming from a team member within the group or outside. In this case we should complete a backend, frontend review holistically as we own the feature domain.

However there are frequently instances where a review request primarily pertains to a non-authentication related development outside of the team, but the review is requested due to an overlap with authentication related features. In such case, we will only review changes to authentication specific files and indicate to the assignee via a comment (mentioning them) that the review is for authentication specific portions only, and they should subsequently request backend or frontend maintainer reviews separately. A similar technique is followed for reviews directed at database reviewers.

Planning

We plan in monthly cycles in accordance with our Product Development Timeline. Our typical planning cycle includes:

- At the beginning of each month, Product creates a planning issue for the coming release.
 - This issue should include the general themes of a release. It should also include placeholders for the quad to put their prioritized feature, bug, and maintenance lists.
 - Issues without estimates should be investigated by engineering throughout the month so weighting does not need to happen at once. Once the prioritized feature list is determined for a milestone, engineering will put a weight on any issues on the list that do not have a weight.
 - Within the first 12 days of the month we estimate the issues. We also identify any key issues that our team feels should be addressed in the upcoming milestone, including bugs, maintenance

tasks, and new features. This approach helps us build a list of important issues that we all agree should be prioritized in future milestones.

- By the 12th day, all planned issues proposed for the next release should be estimated by engineering and put into ~"workflow::ready for development".

- The EM should add any issues that have the potential to slip. Issues that we are sure will slip should be automatically rescheduled to the upcoming release in advance.

- After estimation, Product should make any needed adjustments to the proposed scope based on estimates. At this stage, any issues that the team is committing to deliver should have the ~Deliverable label applied to them by the engineering manager. Critical items such as ~"bug::vulnerability" or others that should be worked on first to allow for a greater chance of completion can be found in the ~Pick-first column of our workflow dashboard such as this one.

Some items may be marked for a certain milestone, but do not have the ~Deliverable label. This means that they have not been committed to for a milestone and should be moved to the backlog or reconsidered for a future release. The Product Manager facilitates this process.

Estimation process

Before work can begin on an issue, we should estimate it first after a preliminary investigation. If the scope of work of a given issue touches several disciplines (docs, design, frontend, backend, etc.) and involves significant complexity across them, consider creating separate issues for each discipline (see an example).

After the PM creates a planning issue for the next release, the quad will gather a prioritized list of issues that should be considered. The EM will then create a "breakdown" issue with a checklist of issues that need to be estimated (example). All Engineers from the group should be assigned to that issue and participate in weighting or breaking down the list.

When estimating development work, please add the appropriate weight to the issue:

| Weight | Description (Engineering) |

| ----- | ----- |

| 1 | The simplest possible change. We are confident there will be no side effects. |

| 2 | A simple change (minimal code changes), where we understand all of the requirements. |

| 3 | A simple change, but the code footprint is bigger (e.g. lots of different files, or tests affected). The requirements are clear. |

| 5 | A more complex change that will impact multiple areas of the codebase, there may also be some refactoring involved. Requirements are understood but you feel there are likely to be some gaps along the way. We should challenge ourselves to break this issue into smaller pieces. |

| 8 | A complex change, that will involve much of the codebase or will require lots of input from others to determine the requirements. These issues will often need further investigation or discovery before being ~"workflow::ready for development" and we will likely benefit from multiple, smaller issues. |

We do not provide estimates greater than 8. If an issue is bigger, we will split the issue or reduce its scope.

If the issue has weight 5 or less, the Engineer adding an estimate should also put the ~"workflow::ready for development" label on it.

If the issue has weight more than 5 (or 5 but it seems it might be split into multiple issues) the Engineer will suggest how to split it. If the Engineer is clear about the splitting they should proactively split the issue themselves, put estimations on the child issues and leave a comment on the parent one. If it results to multiple issues, creating an epic should be considered.

When an Engineer is done with their estimates they should ask another team member to review their weight. This makes it harder to miss a significant aspect of an issue. If the first Engineer is confident (eg. has deep expertise in the problem or the issue is trivial) they can skip this step.

Purpose of Estimation

Our purpose of estimation is to confirm that we

- clearly understand the goal of the issue
- understand what a solution could be, but not necessarily all of the implementation details
- assess whether blockers exist or not

We do not expect that during estimation, implementation details are scrutinized, all blockers are caught, or prove that the proposed solution is the right one. This all happens during development. In development, we, sometimes, reshape the issue, change its solution, or even postpone it. We are proud of doing it.

Considerations for Estimation

With keeping the above in mind, some areas to consider when estimating are:

- If this work needs to be broken down into smaller tasks? Including adding a spike for that investigation/break down.
- Impact on unit, feature and QA testing. Sometimes an apparently small modification in the code can lead to many changes in the tests.
- Is there a hidden inter-dependency between the frontend and backend that needs to be agreed upon?
- Are there data persistence requirements e.g in browser, redis or database?
- How would this feature/bug-fix be deployed and is there complexity in managing the rollout?
- Does this work require collaboration across teams? e.g security

During a release

Engineers should assign issues to themselves based on the current release and ~Deliverable label. We use the following workflow label cycle on our issues:

1. ~"workflow::ready for development" - work is ready to begin, but has not yet started
1. ~"workflow::in dev" - we are looking at the issue, and are either actively investigating, have begun development, or have a draft merge request open
1. ~"workflow::in review" - a merge request has been submitted and reviews have been requested
1. ~"workflow::verification" - the work has merged, and needs to be verified
1. ~"workflow::complete" - the work has been verified by the person assigned to verify; verifier will close the issue and apply label

1. ~"workflow::awaiting security release" - (security MRs only) the work is complete, just pending backports.

Development should not begin on an issue before it's been estimated and given a weight.

Issues with the ~"workflow::verification" or ~"workflow::awaiting security release" labels are, for the purposes of release planning, the same as closed issues, because the dev work is complete.

For work items that span greater than 1 week or are high priority deliverables (such as critical infradev issues or bugs), the assignee for the work aims to provide a brief weekly update on the progress of tasks within the issue being worked on. This is intended to be 2-3 lines highlighting what was accomplished recently, and the next steps. We do this to share the progress transparently with our counterparts but also to share domain knowledge and course correct on implementation details if uncovered after work has started.

Labels and how we use them

We have many labels that can be applied to an issue or merge request. Besides the issue workflow labels above, here are the minimum basic labels to apply to issues and merge requests:

- Type (type::feature, type::bug, or type::maintenance)
- Stage that owns the area (devops::software supply chain security)
- Group that owns the area (group::authentication)
- Specialization (frontend, backend, database, documentation)
- security if the issue is related to application security, and breaking change if this work is considered a breaking change

Working on unscheduled issues

With the combination of our capacity planning (EM) and estimation (IC) processes above, engineers should have free time to work on ~"Stuff that should just work" or other topics that interest them. If an unscheduled issue should really be prioritized, bring it up in a planning issue or ask your manager to reduce your capacity further.

How we prioritize for a release

We have cross-functional prioritization aligned with our prioritization framework. The engineering manager will prioritize type::maintenance issues, the product manager will prioritize type::feature issues, and the software engineer in test will prioritize type::bug issues. From there, we are able to select a ratio of the top issues to be planned for the release by using our cross-functional issue board. Starting 16.5, our target ratio is to plan 60% features, 20% bugs, and 20% maintenance per release. Security issues do not count towards these ratios, but instead take away from the total capacity.

Roadmap planning and t-shirt sizing

At the start of the fiscal year, product and engineering counterparts collaborate over areas we want to focus on. This allows us to prepare issues that will be added to priorities.yml ahead of each milestone, gauge whether we are overcommitting to the year, and transparently share

what the delivery goals are for the upcoming year. As part of this exercise, engineering also performs a high level t-shirt sizing to assist with planning. It's important to highlight that the sizing is high level estimation, based on expected complexity and not the actual length of time taken to complete the work similar to how milestone estimation is done. An example of the scales used would be:

Average story points completed per milestone: 48w. This includes bugs, features, maintenance, and any security vulnerability work.

T-shirt sizing scale:

- Small · < 5w
- Medium · < 8w
- Large · < 16 w
- X-large · 32 w

The roadmap items are then marked with the Small, Medium labels and a priority to ensure, higher priority work is scheduled and appropriately resourced.

Monthly cross-functional dashboard review

We create a monthly issue that is assigned to all counterparts. The issue has to be created manually by an Engineering Manager, but we have an issue template in the Authentication discussion repo.

You can read more about this process on the handbook page.

Although we try to plan as described above, we understand that there is always a risk to the ideal ratios due to unexpected high priority security, infradev, and escalation issues.

Meetings

Although we have a bias for asynchronous communication, synchronous meetings are necessary and should adhere to our communication guidelines. Some regular meetings for Authentication are:

Frequency	Meeting	DRI	Attendees	Topics
Weekly	Group-level meeting (internal only)	Product Manager	PM/EM/UX/SET, optionally engineers	Discuss current or upcoming roadmap topics, bring epics/issues up for attention, provide a current status of priority initiatives

For one-off, topic specific meetings, please always consider recording these calls and sharing them (or taking notes in a publicly available document).

Agenda documents and recordings can be placed in the shared Google drive (internal only) as a single source of truth.

All meetings and 1-1's should have an agenda prepared in advance. If this is not the case, you are not obligated to attend the meeting. Confirm if the meeting should be canceled if there is not an agenda by the start time of the meeting.

Group Members

Auth group members who are part of the Authentication group can be @ mentioned on GitLab with @gitlab-org/software-supply-chain-security/authentication.

The following people are permanent members of the group:

```
{{< engineering/stable-counterparts role="SSCS:Authentication" >}}
```

Dashboards

- Error Budget

Links and resources {links}

```
{{% include "includes/engineering/software-supply-chain-security-shared-links.md" %}}
```

- Milestone retrospectives
- Our Slack channels
 - SSCS:Authentication gsscsauthentication
 - Daily standups gsscsauthenticationdaily
- Issue boards
 - Milestone board
 - Task assignment board

SECTION: engineering/development/sec/software-supply-chain-security/authorization.md

Planning

Our Planning issues are the SSOT of what we're working on now, and what we're working on next. We also have an issue board to view these from a workflow perspective. To maintain the issue lists leadership (EM+PM) will keep the lists triaged.

Workflow

Label	Definition
-------	------------

---	---
-----	-----

~workflow::ready for development	Ready to be picked up by an engineer. ~priority::1 being the highest priority.
----------------------------------	--

~workflow::scheduling or lacking a ~workflow label	Shouldn't be picked up by an engineer and needs to be triaged by leadership. If leadership has no intention on this work being done anytime soon, they should assign the ~Backlog milestone.
--	--

~workflow::refinement	Needs further investigation by engineering. After refinement, an issue should have a weight assigned and the workflow label should be updated to
-----------------------	--

~workflow::scheduling |
| ~workflow::solution validation | Needs Product input before work can begin |
| ~workflow::design | Needs design input to proceed and/or is actively being worked on by UX |

Weekly async issue updates

At the end of every week, each engineer is expected to provide a quick async issue update by commenting on their assigned issues using the following template:

markdown

Async issue update

Current status:

<!-- Please provide a quick summary of the current status (one sentence) -->

Shipping this milestone: <!-- Not confident | Slightly confident | Very confident -->

Scope reduction opportunities: <!-- No | Yes, ... -->

/healthstatus <!-- ontrack | needsattention | atrisk -->

/label <!-- ~"workflow::In dev" | ~"workflow::In review" | ~"workflow::verification"
|~"workflow::blocked" -->

<!-- Please apply a :triangularflagonpost: emoji to this comment. For more information see
<https://gitlab.com/jayswain/automated-reporting> -->

We do this to encourage our team to be more async in collaboration and to allow the community and other team members to know the progress of issues that we are actively working on.

Group Members

Authorization group can be @ mentioned on GitLab with @gitlab-org/govern/authorization.

The following people are permanent members of the group:

{{< engineering/stable-counterparts role="Software Supply Chain Security:Authorization" >}}

Team Meetings

Our group holds synchronous meetings to gain additional clarity and alignment on our async discussions. We aspire to record all of our meetings as our team members are spread across several time zones and often cannot attend at the scheduled time.

We have a weekly team sync meeting with rotating AMER/APAC and EMEA/AMER friendly time slots: Tues 20:00 UTC and Weds 14:30 UTC.

Collaboration

Code Review

Because this group works on components of the application that have a far-reaching impact, we take these extra steps in order to reduce our risk of a production incident:

1. Our team's merge requests should be assigned to another Authorization team member for first review in order to build more institutional knowledge across the team. This review should be done as a reviewer. The Authorization approval counts as the approval matching the role of the Authorization Reviewer, e.g. having a Backend Review from Authorization counts as a Backend Review. Once approved, the Authorization Reviewer should request a review from a Maintainer from the appropriate maintainer category.

1. Authorization related merge requests (those touching custom roles and policies related code) require a review by an Authorization Engineer. This is guarded by using the CODEOWNERS feature of GitLab.

Engineering Refinement

Overview

Engineering refinement is about creating alignment among all team members. To ensure an issue is ready to move into development and that the issue will match everyone's expectations when the work is delivered.

The goal of the refinement process is to:

- Collaborate with stakeholders and domain experts, since often times ~group::authorization is making changes in parts of the product owned by other teams.
- Raise any questions or concerns.
- Outline an implementation plan. This is especially important when backend changes need to unblock frontend work.
- Ensure issue is ready to be worked on.
- Breakdown a big issues into smaller tasks, so they can be picked up by different engineers.
- Assign a weight to the issue or the individual tasks.

How will refinement be conducted?

- We will have a weekly meeting with rotating AMER/APAC and EMEA/AMER friendly time slots. Attendance is optional. The calendar will act as a reminder for team members to work on refinement weekly. These meetings can also promote candid discussion for other ongoing issues that engineers are facing.
- The purpose of these meetings will be to refine the issues that have been assigned the workflow::refinement label.
- If an issue requires further research, an engineer can conduct more research outside of the meeting. In a later meeting, the more refined issues can then be estimated.
- If an engineer is conducting refinement async, they can follow the steps listed below, summarize findings (and proposals if you have any) and mention other team members and involved counterparts.
- If it seems unlikely to reach an agreement during the opened discussion or there is a clear misunderstanding, it can be added as an agenda to the sync meeting. Engineers can optionally also those who might be able to help because of their expertise.

Steps

- Ensure the issue is fully understood.
- Check the issue description for completeness.
 - Does it have the necessary designs?
 - Is the expected functionality clearly articulated?
 - Are there dependencies? On other teams or on other issues?
- If the issue is not complete:
 - Tag @jrandazzo (PM) or @jswain (EM), so they know that the item can not be fully groomed. They can help complete the issue and outline what is needed.
- Break down the issue into tasks.
 - If the issue requires both frontend and backend work, then we can create different tasks to divide that work. This will allow different engineers to collaborate on the issue together.
 - If it's a new or complicated feature, specify what can be delivered in the first iteration, and what can be delivered in future iterations.
- Outline an implementation plan
 - If the issue requires both frontend and backend work, define the request/response structure of API endpoints to ensure there is minimal computation in the Vue components.
 - If further research is needed to figure out the implementation plan, a spike issue can be created. We can circle back to the issue once we have the findings from the spike.
- Assign a weight using t-shirt sizing approach, set the label ~workflow::scheduling and tag @jswain and @jrandazzo. If the issue has been broken down into tasks, weights can be assigned to each of the tasks.
- Once the next steps are clear, update the description of the issue(s) and unassign yourself.

Links and resources {links}

- Our Slack channels
 - Govern:Authorization ggovernauthorization

SECTION: engineering/development/sec/software-supply-chain-security/compliance.md

Mission

The Compliance group's mission is to provide visibility into an organizations compliance posture within GitLab and provide tools for organization's to identify non-compliant activities and scope enforcement of compliance requirements.

What we work on

- We use the Group Direction page to describe our high-level goals and direction for our group.
- From the high-level goals and direction we filter down to a prioritised list of Epics, we try to keep updated in our Tactical Priorities
- This prioritised list we then use when planning each Milestone. Each Milestone will have its own Issue in our Planning Epic
 - In addition to using the high-level goals and direction as an input to planning Milestones, the Compliance Product Manager considers input from Sales, customers, and internal stakeholders (dogfooding) to decide on the priority for the issues added to each Milestone.

- We also use OKRs to help prioritise strategic initiatives within the group. We use Issues for planning and collate them in our OKR Epic

Top Priorities FY25

- The biggest priority for FY25 is to continue to integrate Compliance Frameworks with Security Policies. Expanding the abilities within the Compliance Center and moving towards removal of Compliance Pipelines.
- Continue expanding on the Adherence Report by adding more Standards/Checks, while also integrating violations as the other side of the compliance posture (enforcement vs violation).
- We are also looking at building customizable checks and the highest priority standards for our customers.
- Rollout full usage of ClickHouse as the DB for audit events. Unblocking features to give better insights into compliance across the GitLab platform.
- Provide a standard way for adding new Streaming Audit Event destination types and filtering, then increasing our streaming audit event offering through more 3rd party integrations and filtering options.
- Create a platform for easily allowing contributions outside the Compliance group to add audit events throughout the product. This includes planning and standardizing all current open audit event issues, creating easily consumable tutorials and guidelines, and potentially creating an audit event day (similar to pajamas migration days but for backend).

How we work

- In accordance with our GitLab values.
- Transparently: nearly everything is public, we record/livestream meetings whenever possible (see links)
- We get a chance to work on the things we want to work on.
- Everyone can contribute; no silos.
 - The goal is to have product give engineering and design the opportunity to be involved with direction and issue definition from the very beginning.
- We do an optional, asynchronous daily stand-up in our stand-up channel:
 - Software Supply Chain Security:Compliance gsscscomplianceupdates

Code Review

Because this group works on components of the application that have a far-reaching impact, we take these extra steps in order to reduce our risk of a production incident:

1. To build more institutional knowledge across the team we try to assign our merge requests to another Compliance team member for first review.
1. Compliance related merge requests require a review by a Compliance Engineer. This is guarded by using the CODEOWNERS feature of GitLab.

Feature Flags

1. Compliance merge requests use feature flags where it makes sense to reduce risk. We follow the Feature Flag Lifecycle as closely as possible. For extended or multi-merge request development, all work should remain behind a dedicated feature flag. This approach allows incremental changes to be merged safely, keeping incomplete functionality off for production

users.

1. If a feature flag is used, a feature flag rollout plan issue should be created and added to the appropriate epic. Support (supportgitlab-com) will also be notified if necessary.

1. Once the feature is ready for testing, the directly responsible individual (DRI) should enable the feature flag on the staging environment and announce availability to relevant stakeholders (PM, EM, designer, wider team) via Slack and commenting in the rollout issue. This ensures prompt feedback, thorough validation, and keeps everyone in the loop.

1. After validating the feature in staging, enable the feature flag in production in accordance with your rollout plan. If any issues arise, disabling the feature flag provides a quick rollback mechanism without the need to revert code. This controlled approach aligns with GitLab's best practices for delivering new features safely and efficiently.

Working on ad hoc work and questions

From time to time, there will be ad hoc work and questions that arise, such as Slack questions, questions in Issues, Error Budget investigations, etc. All Compliance group members are encouraged to watch these mediums and engage.

As first responder, we will acknowledge the ad hoc work / question in the appropriate medium. This is to ensure that the questioner knows that we are on it.

Similar to Spikes, as a rule of thumb, if the work will take longer than 1 hour to investigate and respond, then create an issue. This is to ensure that this work is accounted for, is transparent and has a DRI. This can be done through asking the questioner to create an issue, or taking ownership and creating the issue ourselves.

We try to ensure that the ad hoc work Issue has as much info as possible, asking for more info where required. Before working on an Issue, we make sure to define a clear question or problem that needs to be delivered.

Next, we take ownership of the Issue, assigning it to ourselves. We also add the :eyes: reaction to the Slack message/Issue comment to indicate that investigation has begun. We also ensure correct label hygiene (~type::, ~group::, ~priority::, ~workflow::). In the comments, ping the Compliance EM and PM to ensure transparency. Depending on priority, it may need to go through cross-functional prioritization process to get it planned, scheduled, and completed.

Once the investigation is complete, we follow up in the original medium. We also add the :whitecheckmark: reaction to the original Slack message/Issue comment to indicate that investigation is complete.

Spikes

We use spikes to conduct research, prototype, investigation to gain knowledge, reduce the risk of a technical approach, and better understand a requirement.

When we identify the need for a spike, we will create a new issue, clearly label it as such example, conduct the spike, and document the findings in the spike issue.

Spikes are great for accounting for ad-hoc work and questions that can come through during a Milestone. A rule of thumb, if the work will take longer than 1-2 hours, then create a spike issue.

Spike issues, like any other issue, will go through the cross-functional prioritization process to get it planned, scheduled, and completed.

Before working on a spike we make sure to clearly define:

1. A time-box of how much time to spend on the issue.
1. A clear question or problem that needs to be delivered.

Proof-of-concept MRs

We strongly believe in iteration and delivering value in small pieces. Iteration can be hard, especially when you lack product context or are working in a particularly risky/complex part of the codebase. If you are struggling to estimate an issue or determine whether it is feasible, it may be appropriate to first create a proof-of-concept MR. The goal of a proof-of-concept MR is to remove any major assumptions during planning and provide early feedback, therefore reducing risk from any future implementation.

- Create an MR, prefixed with PoC:.
- Explain what problem the PoC MR is trying to solve for in the MR description.
- Timebox it. Can you determine feasibility or a plan in less than 2-3 days?
- Identify a reviewer to provide feedback at the end of this period.
- Close the MR. Provide a summary in the original issue on what you learned from the PoC, including product and performance implications.
 - State whether you are able to move forwards with implementation or not.
 - Please do not close the issue.

The need for a proof-of-concept MR may signal that parts of our codebase or product have become overly complex. It's always worth discussing the MR as part of the retrospective so we can discuss how to avoid this step in future.

Working on unscheduled issues

Everyone at GitLab has the freedom to manage their work as they see fit, because we measure impact, not activity. Part of this is the opportunity to work on items that aren't scheduled as part of the regular monthly release. This is mostly a reiteration of items elsewhere in the handbook, and it is here to make those explicit:

1. We expect people to be managers of one, and we use GitLab ourselves. If you see something that you think
 - is important, you can request for it to be scheduled, or you can work on a proposal yourself, as long as you keep your other priorities in mind.
1. From time to time, there are events that GitLab team-members can participate in, like the issue bash. Anyone is welcome to participate in these.

When you pick something to work on, please:

1. Follow the standard workflow and assign it to yourself.
1. Share it in Slack to encourage transparency

Testing

We aim to uphold GitLab's principle of fostering an environment where Quality is everyone's responsibility.

Testing is an essential part of our product development workflow and code review process.

Information regarding test coverage can be found via these issues/epics:

- Unit test coverage of Compliance features
- Shift-left coverage provided by E2E tests
- Increasing E2E test coverage

Milestone Planning

We plan in monthly cycles in accordance with our Product Development Timeline. Our typical planning cycle is suggested to look like:

Pre-planning

- By the 4th, Product should have created a planning issue for their group in the Compliance project for the coming release using the template.
- The Compliance quad will add a tentative plan for the release, outlining the highest priority issues within each of their respective areas.
- We prioritize using the cross-functional prioritization. The Product Manager will prioritize type::feature issues, the Engineering Manager will prioritize type::maintenance issues, and the Quality Manager will prioritize type::bug issues.
- Pre-planning is completed asynchronously by Product, Engineering, Quality and Design on the issue, this is to identify any unknowns or questions that need to be answered and resolved prior to final planning.
- Issues can be either in Refinement phase (workflow::planning breakdown and workflow::solution validation) or Implementation phase (workflow::scheduling and workflow::ready for development).
- Engineering Manager will update issue with the estimated group capacity for the Milestone
 - To assist with capacity planning, we track the cumulative weight of closed issues over the past 3 releases on a rolling basis. The proposed scope of work for the next release should not exceed 80% of this to account for slippage from the previous release.

Planning

- To review the proposed scope and kick start estimation, there is a synchronous meeting with Engineering and Design on the second Friday of the Month.
- From there, we are able to select a ratio of the top issues to be planned for the milestone by using our cross-functional issue board. Our target ratio is to plan 60% features, 30% maintenance, and 10% bugs per milestone. The data below helps us understand our overall cross-functional status.

- Each area will include a 20% stretch target of issues. EG capacity 40W x 60% features = 24W + 20% = ~28W
- With a list now determined, Milestones and label workflow::ready for development can be added to this work, ready for the Milestone to start.
- Engineering Manager will comment and add label to the assigned issues with clear expectations for the Milestone using one of four scoped label types:
 - goal::planning - full implementation plan created and if needed reviewed and approved
 - goal::development - development has begun and either has in dev or in review workflow
 - goal::complete - development complete and issue verified
 - goal::stretch - added as a stretch goal, no expectation to start
- We try to plan 1-2 Milestones ahead, we include a max of 2 planning issues (workflow::planning breakdown and workflow::solution validation) per person at the start of a Milestone, this is a rule of thumb.
 - When a planning issue is included in a Milestone it is also assigned to team members. This is to provide clarity on what and who is doing what planning in the Milestone.
- By the 20th, Product should review the release that just concluded development (currently, we transition development work from one release to the next on the 18th) for issues that slipped from the milestone. Please evaluate issues that weren't merged in time and reschedule them appropriately.
- Identify any issues which may have security implications, and ping the Application Security Stable Counterpart and/or request an Application Security Review. The Product Manager will list these in the planning issue.

Issue Prioritization

Our priorities should follow overall guidance for Product. This should be reflected in the priority label for scheduled issues:

Priority	Description	Probability of shipping in milestone
priority::1	Urgent: top priority for achieving in the given milestone. These issues are the most important goals for a release and should be worked on first; some may be time-critical or unblock dependencies.	~100%
priority::2	High: important issues that have significant positive impact to the business or technical debt. Important, but not time-critical or blocking others.	~75%
priority::3	Normal: incremental improvements to existing features. These are important iterations, but deemed non-critical.	~50%
priority::4	Low: stretch issues that are acceptable to postpone into a future release.	~25%

Weekly Progress Updates

- Engineering manager (EM): Posts a comment on the current planning issue on the first day of the week, to prompt team members to update.
 - Format: | NUMBER | EPIC | GOAL | DRI |
- Implementing team member: Posts a short update (1-2 sentences) in comment thread on each item where you are the directly responsible individual (DRI). The update should be in reference to the milestone goal set out during planning. If there is nothing to report omit the item or use "see previous update"

Intention of this is to:

- Have a reference for verbal weekly updates during the Compliance Weekly meetings.
- Document the weekly update for reporting.
- Remove the on-the-spot/ad-hoc nature of the weekly updates.
- Not be a personal performance indicator. The DRI is not necessarily the only individual working on an effort, and there may always be a change in priorities or other factors such as personal time off (PTO) or individuals being out of office (OOO).

During Milestone

- When an issue is introduced into a release after Kickoff, an equal amount of weight must be removed to account for the unplanned work.
- Development should not begin on an issue before it's been estimated and given a weight.
- By the 15th, engineering merge requests should be merged. In other words, we assume code merged after the 15th will not be in the release. That allows time for the release to be finalized, and any associated Release Posts to be merged by the 17th. (This is an experiment starting with 13.11.)

Development Flow

We generally follow the Product Development Flow:

1. workflow::problem validation - needs clarity on the problem to solve
1. workflow::design - needs a clear proposal (and mockups for any visual aspects)
1. workflow::solution validation - needs refinement and acceptance from engineering
1. workflow::planning breakdown - needs a Weight estimate
1. workflow::scheduling - needs a milestone assignment
1. workflow::ready for development
1. workflow::in dev
1. workflow::in review
1. workflow::verification - code is in production and pending verification by the DRI engineer
1. workflow::complete - the work has been verified and the work is complete, the issue should be closed at this stage

Generally speaking, issues are in one of two states:

- Discovery/refinement (1-4): we're still answering questions that prevent us from starting development,
- Implementation (6-9): an issue is waiting for an engineer to work on it, or is actively being built.

While individual groups are free to use as many stages in the Product Development Flow workflow as they find useful, we should be somewhat prescriptive on how issues transition from discovery/refinement to implementation.

Discovery/Refinement

The end goal is defined, where all direct stakeholders says "yes, this is ready for development". Some issues get there quickly, some require a few passes back and forth to figure

out.

The goal is for engineers to have buy-in and feel connected to the roadmap. By having engineering included earlier on, the process can be much more natural and smooth. To do so, engineering managers, engineers, and designers can be pinged directly from the issue.

To move to the implementation phase all issues should have an Implementation Plan and a Weight

Backlog

Backlog management is very challenging, but we try to do so with the use of labels and milestones.

To identify issues that need refinement, use the "Next Up" label.

The purpose of the "Next Up" label is to identify issues that are currently in any workflow stage before workflow::ready for development. By using this "Next Up" label in addition to workflow labels, we're able to see exactly what is being refined, e.g., problem, design, solution. This helps identify which issues are closer to being ready to schedule.

Issues shouldn't receive a milestone for a specific release (e.g. 13.0) until they've received a · from both Product and Engineering. This also means the issue should not be labeled as workflow::ready for development.

- Product approval is represented by an issue moving into workflow::planning breakdown.
- Engineering approval is represented by an issue weight measuring its complexity.

Estimation

Before work can begin on an issue, we should estimate it first after a preliminary investigation.

If the scope of work of a given issue touches several disciplines (docs, design, frontend, backend, etc.) and involves significant complexity across them, consider creating separate issues for each discipline (see an example).

Issues without a weight should be assigned the workflow::planning breakdown label.

When estimating development work, please assign an issue an appropriate weight:

| Weight | Description (Engineering) |

| ----- | ----- |

| 1 | The simplest possible change. We are confident there will be no side effects. |

| 2 | A simple change (minimal code changes), where we understand all of the requirements. |

| 3 | A simple change, but the code footprint is bigger (e.g. lots of different files, or tests affected). The requirements are clear. |

| 5 | A more complex change that will impact multiple areas of the codebase, there may also be some refactoring involved. Requirements are understood but you feel there are likely to be some gaps along the way. We should challenge ourselves to break this issue in to smaller pieces. |

| 8 | A complex change, that will involve much of the codebase or will require lots of input from others to determine the requirements. These issues will often need further investigation

or discovery before being workflow::ready for development and we will likely benefit from multiple, smaller issues. |

| 13 | A significant change that may have dependencies (other teams or third-parties) and we likely still don't understand all of the requirements. It's unlikely we would commit to this in a milestone, and the preference would be to further clarify requirements and/or break in to smaller Issues. |

As part of estimation, if you feel the issue is in an appropriate state for an engineer to start working on it, please add the workflow::ready for development label. Alternatively, if there are still requirements to be defined or questions to be answered that you feel an engineer won't be able to easily resolve, please add the workflow::blocked label. Issues with the workflow::blocked label will appear in their own column on our planning board, making it clear that they need further attention. When applying the workflow::blocked label, please make sure to leave a comment and ping the DRI on the blocked issue and/or link the blocking issue to raise visibility.

Implementation Plan

For engineers, you may want to create an implementation approach when moving an issue out of ~workflow::planning breakdown. A proposed implementation approach isn't required to be followed, but is helpful to justify a recorded weight.

As the DRI for workflow::planning breakdown, consider following the example below to signal the end of your watch and the issues preparedness to move into scheduling. While more straightforward issues that have already been broken down may use a shorter format, the plan should (at a minimum) always justify the "why" behind an estimation.

The following is an example of an implementation approach from <https://gitlab.com/gitlab-org/gitlab/-/issues/247900implementation-plan>. It illustrates that the issue should likely be broken down into smaller sub-issues for each part of the work:

md

Implementation approach

~database

1. Add new mergerequestsauthorapproval column to namespacesettings table (The final table is TBD)

~"feature flag"

1. Create new groupmergerequestapproversrules flag for everything to live behind

~backend

1. Add new field to ee/app/services/ee/groups/updateservice.rb:117

1. Update ee/app/services/ee/namespacesettings/updateservice.rb to support more than just one setting

1. (if feature flag enabled) Update the Projects::CreateService and Groups::CreateService to update newly created projects and sub-groups with the main groups setting

1. (if feature flag enabled) Update the Groups API to show the settings value

1. Tests tests and more tests :muscle:

- In particular, cover both happy and unhappy paths, and consider tests for scenarios that could result in false positives or negatives

~frontend

1. (if feature flag enabled) Add new Merge request approvals section to Groups general settings

1. Create new Vue app to render the contents of the section

1. Create new setting and submission process to save the value

1. Tests tests and more tests :muscle:

- In particular, cover both happy and unhappy paths, and consider tests for scenarios that could result in false positives or negatives

~documentation

1. Update docs page eg <https://docs.gitlab.com/ee/administration/auditeventreports.html>

1. Update the GraphQL examples <<https://gitlab.com/gitlab-org/software-supply-chain-security/compliance/engineering/graphql-example-requests>>

~quality

1. Add new group-level end-to-end test based on existing project-level end-to-end test (include the path to the existing test eg path/to/existingtest)

The DRI will ping a relevant counterpart (Quality, UX, etc) and domain expert (database, backend, frontend) before moving the issue to workflow::scheduling. This gives the domain expert the opportunity to approve the implementation plan or raise any potential pitfalls or concerns before work begins.

For domain expert review of development implementation plan, in case of trivial changes, the approval can be solicited from any of the relevant compliance development team members. Do try to find a person who has context around the topic. In case of non-trivial changes, opinions from the whole relevant compliance backend or frontend or both team members should be solicited by tagging respective group (@gitlab-org/software-supply-chain-security/compliance/engineering) in the issue's comment. Deciding whether the implementation is trivial or non-trivial depends on the discretion of DRI and the initial domain expert asked for review.

Once an issue has been estimated, it can then be moved to workflow::scheduling to be assigned a milestone before finally being workflow::ready for development.

Breaking down or promoting issues

Depending on the complexity of an issue, it may be necessary to break down or promote issues. A couple sample scenarios may be:

- We need to do discovery on the design, before we do anything else. A "Discovery:" issue may work best here as it helps to contain the design thinking and discussion there, with the end result being transferred over to a "Implementation:" issue. These prefixes also help to

organize what type of issue they are, in the case they are linked to parent issues or epics.

- The scope of work is larger than anticipated, and needs to be broken down further, e.g., it currently has a weight higher than 5. It may suit you to then promote said issue to an epic, to break it down into smaller issues to list out the different iterations or phases of work that need to happen to deliver the overall feature that was originally proposed.
- The scope of work is clear, but a bit unwieldy for one issue. It may make sense to keep the given issue as is, to keep the conversation and activity visible to everyone, but create separate child design, backend, or frontend issues to track the more nuanced progress of a given issue.

If none of the above applies, then the issue is probably fine as-is! It's likely then that the weight of this issue is quite low, e.g., 1-2.

Implementation

Verification

The issue verification should be done by someone else other than the issue assignee. This decreases the case of defects getting into production and a different perspective to cover more test cases.

The verification process is also about sharing knowledge of development between the rest of the group. In an all remote organisation it is easy to become siloed developing features even in the same team, and this attempts to help bridge this.

- Before moving the issue to workflow::verification, the issue assignee must verify the changes on production themselves to make sure the feature is working as expected.
- All MRs should have verification steps in the description. In the case where multiple MRs are created for an issue, the engineer who is assigned to the issue should add complete verification steps in the issue description or as a reply to the triage bot's comment.
- When an engineer has merged their work, they should move their issue into the verification status, indicated by the ~workflow:verification label and wait until they receive notification that their work has been deployed on staging via the release issue email.
 - For ~type::bug, ~type::feature, or big changes the engineer should verify again once the change is available on .com/production and leave a comment summarizing the testing that was completed. Also provide a link to a project or page, if applicable.
- Issues in the ~workflow:verification state are assigned randomly by the triage bot based on the verification policy to an applicable team engineer. This engineer should then verify the issue. In addition to following the verification steps provided, the verifying engineer is encouraged to do exploratory testing by using the feature in a way that doesn't follow the happy path.
- Once the issue has been verified in production, the verifying engineer will close the issue and unassign themselves. The issue will automatically get the ~workflow::complete label added.

In cases where verification in staging or production is unfeasible, the staging-ref environment may be used. For complex setups, the DRI for the MR should work with a domain expert to ensure verification steps are clear and correct.

In some cases it may be appropriate for the MR author to verify the change themselves, for example feature flag rollout or monitoring a background migration. In these cases, the author

should add the verified-by-author label to the issue to prevent the triage bot from assigning another team member, and the author should add a comment in the issue explaining why they are verifying the issue themselves.

Handling Discoveries of Issues

Verifier: the engineer verifying the issue on .com/production (not the MR author)

1. Verifier: documents findings by commenting on the verification thread on the issue.
1. Verifier: opens new issues ~"type::bug" or ~"type::feature" based on findings.
 1. Verifier: sets the severity and/or priority based on priority/severity triage process and the issue type (~"type::bug" or ~"type::feature").
 1. ~"severity::1" / ~"severity::2" to be pulled directly into the milestone with comment added to ping(@) the engineering manager.
 1. Verifier: assigns the MR author to newly opened issues.
 1. Verifier: closes the issue and unassigns themselves. The issue will automatically get the ~workflow::complete label added.

Demos

During the workflow::verification process, we determine whether the Issue requires a demo. If unsure, work with PM to determine if a demo is required. Demos are great for showcasing progress and help users quickly understand how to use a features and its benefits. Our process for this is similar to Single Engineer Groups Demo:

- Record a video on your computer or via Zoom livestream of the working software. You can also use loom for recording demos.
- Upload to the GitLab Unfiltered Channel and add to the Compliance Group - Product Playlist
- Post the YouTube link to the appropriate Slack channel and in the Issue

Release posts

For issues which need to be announced in more detail, a release post can be automatically created using the issue.

When working on an issue, either in planning, or during design and development, you can use the release post item generator to have the release post created and notify all the relevant people.

If you do not want an issue to have a release post, make sure that the issue does not have a release notes section or use a release post item:: label.

Meetings

Although we have a bias for asynchronous communication, synchronous meetings are necessary and should adhere to our communication guidelines. Some regular meetings that take place in Compliance are:

Frequency	Meeting	DRI	Possible topics
-----	-----	-----	-----

-----|
 | Weekly (alternating between APAC/EMEA and AMER) | Group-level meeting |
 Engineering Manager | Ensure current release is on track by walking the board, unblock specific
 issues |
 | Monthly | Planning meeting | Product Manager | See Planning
 section |

For one-off, topic specific meetings, please always consider recording these calls and sharing them (or taking notes in a publicly available document).

Agenda documents and recordings can be placed in the shared Google drive (internal only) as a single source of truth.

Meetings that are not 1:1s or covering confidential topics should be added to the Software Supply Chain Security Shared calendar.

All meetings should have an agenda prepared at least 12 hours in advance. If this is not the case, you are not obligated to attend the meeting. Consider meetings canceled if they do not have an agenda by the start time of the meeting.

Planning for PTO

We follow the Engineering process for taking time off and GitLab team members Guide to Time Off.

Group Shared Calendar

The Compliance Group Shared Calendar is used to make sure PTO events are visible to everyone on the team.

Syncing your Time Off by Deel entries to the Shared Calendar

Below are the steps to add the calendar to Time Off by Deel:

1. Open Slack

1. Open app in slack "Time Off by Deel"

1. Go to tab "Home",

1. Click on the dropdown "Your Events", and select "Calendar Sync".

1. Under "Additional calendars to include?", click on "Add calendar".

1. Add the following calendar ID:

ce21c4d99155603fa2f3b06f41628bebf6013218000c822ec2fdfe7ed877d3e1@group.calendar.google.com

Great job! · Your PTO events will be synced to Compliance Group Shared Calendar from now on. ·

Group News

The EM will usually create a general update for the group on what is happening within the company and within the group on a weekly basis.

This update currently takes the form of an issue within the compliance update Epic

The Compliance EM also contributes to issues in the Software Supply Chain Security stage weekly updates epic.

Group Members

The following people are permanent members of the group:

```
{{< engineering/stable-counterparts role="Software Supply Chain Security.+Compliance" >}}
```

Links and resources {links}

- GitLab
 - gitlab-org/software-supply-chain-security/compliance
 - General issues and discussions
 - Engineering issues and discussions
 - Compliance alias: [@gitlab-org/software-supply-chain-security/compliance](https://gitlab-org/software-supply-chain-security/compliance)
 - Compliance engineering alias: [@gitlab-org/software-supply-chain-security/compliance/engineering](https://gitlab-org/software-supply-chain-security/compliance/engineering)
 - Milestone retrospectives
 - Issue boards
 - Build board
 - Planning board
- Our Slack channels
 - Software Supply Chain Security:Compliance [gsscscompliance](#)
 - Daily standups [gsscscomplianceupdates](#)
 - Group related sentry errors [gsscscompliancealerts](#)
 - Compliance engineering alias: [@sscscompliance](#)
- Google Group
 - [sec-software-supply-chain-security-compliance](#)
- Compliance group playlists on GitLab Unfiltered channel
 - Product
 - Meetings

```
{{% include "includes/engineering/software-supply-chain-security-shared-links.md" %}}
```

```
## SECTION: engineering/development/sec/software-supply-chain-security/pipeline-security/_index.md
```

Vision

We provide security in the software supply chain process, ensuring the right level of visibility to pipelines and jobs. This enables organizations to operate their CI pipelines securely.

Our team is responsible for delivering on the following categories:

- Secrets Management

- Artifact Security

The current and planned category maturity can be found on the Maturity page.

Mission

Our mission is to give organizations features which enable secure pipeline operations, in an operationally efficient and highly secure way at scale.

Performance Indicators

We measure the value we contribute by using a Product Performance Indicator. One of the PI process goals is to ensure that, as a product team, we are focused on strategic and operational improvements to improve leading indicators, precursors of future success.

<!-- Our current PI for the Pipeline Security group is still to be determined. View Sec Section performance indicators (internal handbook).-->

Team Members

The following people are permanent members of the Software Supply Chain Security:Pipeline S